МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

ИНСТИТУТ ИНФОРМАЦИОННЫХ СИСТЕМ И ЗАЩИТЫ ИНФОРМАЦИИ

КАФЕДРА № 43

ОТЧЕТ ЗАЩИЩЕН С ОЦЕНКОЙ	ПРЕПОДАВАТЕЛЬ	ассистент А. Э. Зянчурин
	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 3

«Интеграция общего (системного) и специального программного обеспечения в программных комплексах»

по дисциплине: «Основы программной инженерии»

Индивидуальный раздел: «Проектирование, оптимизация и администрирование базы данных PostgreSQL: схема данных, безопасность, нагрузка 100 тыс. строк, бэкапы»

РАБОТУ ВЫПОЛНИЛ	СТУДЕНТ гр. № 4434	Кочетков Р. А.
	подпись, дата	инициалы, фамилия

Санкт-Петербург

2026

ЗАДАНИЕ

На индивидуальное выполнение раздела лабораторной работы студентом группы 4434 Кочетков Р. А..

Тема задания: Проектирование, оптимизация и администрирование базы данных PostgreSQL супермаркета в роли инженера баз данных

Перечень подлежащих исследованию, проектированию и разработке вопросов:

— Физическое проектирование реляционной схемы данных PostgreSQL 17 по варианту 36 (таблицы products, stock_movements, receipts, receipt_items, app_users, sessions).

— Реализация механизма сохранения исторических цен: разделение актуальной цены в карточке товара и зафиксированной цены продажи в строке чека.

— Разработка ограничений целостности и триггерных функций: запрет модификации оплаченного чека, контроль неотрицательности остатков, пессимистические блокировки FOR UPDATE.

— Проектирование ролевой модели безопасности СУБД: разграничение полномочий ролей market_admin (DDL), market_app (DML без DDL) и market_reader (SELECT-only), отзыв прав роли PUBLIC и настройка сетевого фильтра pg_hba.conf.

— Проектирование стратегии индексов (B-Tree индексы по штрихкодам, внешним ключам и курсорам пагинации) и оптимизация под нагрузку до 100 000 объектов.

— Генерация нагрузочного массива данных (100 тыс. товаров, 100 тыс. чеков, 500 тыс. строк) и профилирование планов запросов утилитой EXPLAIN (ANALYZE, BUFFERS).

— Регламент и практическая проверка создания логических резервных копий (pg_dump) и восстановления в изолированную базу (pg_restore) с аудитом контрольных сумм.

Задание выдал: _____ / А. Э. Зянчурин /
Подпись: _____
Задание принял: _____ / Кочетков Р. А. /
Дата выдачи: «____» _____ 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ

1 ФИЗИЧЕСКАЯ СХЕМА БАЗЫ ДАННЫХ И ЦЕЛОСТНОСТЬ ДАННЫХ

1.1 Состав таблиц и связи реляционной схемы

1.2 Сохранение неизменности исторических цен продаж

1.3 Ограничения целостности и триггеры СУБД

2 БЕЗОПАСНОСТЬ И РОЛЕВАЯ МОДЕЛЬ ДОСТУПА В POSTGRESQL

2.1 Разграничение полномочий системных ролей СУБД

2.2 Ограничение привилегий роли PUBLIC и сетевая изоляция

3 ИНДЕКСИРОВАНИЕ, ОПТИМИЗАЦИЯ И РАБОТА С 100 ТЫСЯЧАМИ ОБЪЕКТОВ

3.1 Стратегия создания индексов

3.2 Генерация нагрузочного массива данных

3.3 Профилирование планов запросов EXPLAIN (ANALYZE, BUFFERS)

3.4 Оценка производительности каталога и емкости табличных пространств

4 РЕГЛАМЕНТ РЕЗЕРВНОГО КОПИРОВАНИЯ И АВАРИЙНОГО ВОССТАНОВЛЕНИЯ

4.1 Процедура создания логических дампов pg_dump

4.2 Процедура восстановления базы утилитой pg_restore

4.3 Проверка целостности и совпадения записей после восстановления

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

ПРИЛОЖЕНИЕ А Листинг анализа планов выполнения запросов explain.txt

ПРИЛОЖЕНИЕ Б Протоколы нагрузочного тестирования и восстановления базы

ВВЕДЕНИЕ

Настоящий отчет отражает результаты проектирования, оптимизации и администрирования реляционной базы данных PostgreSQL 17 для информационной системы супермаркета (вариант 36) в рамках лабораторной работы № 3 по курсу «Основы программной инженерии».

База данных является фундаментом надежности информационной системы розничной торговли. На СУБД возлагаются задачи обеспечения транзакционной целостности по стандарту ACID, безопасного разграничения прав доступа, эффективного индексирования больших объемов номенклатурных справочников (до 100 тыс. записей), а также гарантия сохранности информации посредством регламентного резервного копирования.

Проектирование выполнено в соответствии с методическими требованиями [1], стандартами ГОСТ 2.105-2019 и ГОСТ 7.32-2017 [2, 3].

1 ФИЗИЧЕСКАЯ СХЕМА БАЗЫ ДАННЫХ И ЦЕЛОСТНОСТЬ ДАННЫХ

1.1 Состав таблиц и связи реляционной схемы

Физическая схема базы данных реализована в третьей нормальной форме (3NF) и включает следующие таблицы:

— products — справочник товаров (id BIGSERIAL PRIMARY KEY, barcode VARCHAR(13) UNIQUE, name TEXT, price NUMERIC(10,2), stock INTEGER, category TEXT, is_archived BOOLEAN);

— stock_movements — журнал складских проводок (id BIGSERIAL PRIMARY KEY, product_id BIGINT REFERENCES products(id), quantity_change INTEGER, reason TEXT, created_at TIMESTAMPTZ);

— receipts — кассовые чеки (id BIGSERIAL PRIMARY KEY, cashier_id BIGINT REFERENCES app_users(id), status VARCHAR(20), total_amount

NUMERIC(10,2), cash_tendered NUMERIC(10,2), created_at TIMESTAMPTZ, paid_at TIMESTAMPTZ);

— receipt_items — спецификация чеков (receipt_id BIGINT REFERENCES receipts(id) ON DELETE CASCADE, product_id BIGINT REFERENCES products(id), quantity INTEGER, unit_price NUMERIC(10,2), PRIMARY KEY (receipt_id, product_id));

— app_users и sessions — учетные записи сотрудников, хеши scrypt, роли доступа и токены авторизации.

1.2 Сохранение неизменности исторических цен продаж

Для предотвращения искажения финансовой отчетности в схеме жестко разделены две сущности цены:

— products.price — текущая розничная цена товара в каталоге, которую менеджер может корректировать в любой момент времени;

— receipt_items.unit_price — историческая цена единицы товара, зафиксированная в момент добавления позиции в чек. Последующие изменения базовой цены в таблице products не влияют на закрытые чеки.

1.3 Ограничения целостности и триггеры СУБД

На уровне ядра СУБД внедрены декларативные ограничения и процедурные триггеры:

— CHECK (price > 0) и CHECK (stock >= 0) — запрет отрицательных цен и товарных остатков;

— CHECK (status IN ('draft', 'paid', 'cancelled')) — контроль допустимых состояний чека;

— Триггер trg_protect_paid_receipts — при попытке выполнить UPDATE или DELETE для чека в статусе 'paid' генерирует исключение EXCEPTION 'Modification of paid receipts is strictly prohibited';

— Триггер trg_check_receipt_items — запрещает добавлять или модифицировать строки в чеке, если чек не находится в статусе 'draft'.

2 БЕЗОПАСНОСТЬ И РОЛЕВАЯ МОДЕЛЬ ДОСТУПА В POSTGRESQL

2.1 Разграничение полномочий системных ролей СУБД

В PostgreSQL реализована многоуровневая ролевая модель (таблица 1), исключающая работу прикладных сервисов под учетной записью суперпользователя postgres:

— market_admin — роль администратора с правами создания схемы и таблиц (DDL), запуска начальных миграций;

— market_app — сервисная роль веб-приложения с правами выполнения DML-операций (SELECT, INSERT, UPDATE). Роли запрещены команды DROP TABLE, ALTER TABLE и удаление строк чеков;

— market_reader — роль аудитора исключительно с правами чтения (SELECT-only).

Таблица 1 — Ролевая модель доступа в СУБД PostgreSQL

Роль	Тип	Права
market_owner	NOLOGIN	Владелец схемы, таблиц и функций
market_admin	LOGIN, член market_owner	Управление схемой и начальными данными; не суперпользователь кластера
market_app	LOGIN	Чтение нужных таблиц, вставка карточек/чеков/движений/сеансов, обновление разрешённых столбцов, удаление сеансов
market_reader	NOLOGIN	Групповые права чтения товаров, чеков, строк и движений без хешей паролей

2.2 Ограничение привилегий роли PUBLIC и сетевая изоляция

В целях безопасности выполнен отзыв стандартных привилегий роли PUBLIC:

— REVOKE ALL ON SCHEMA public FROM PUBLIC;

— REVOKE CONNECT ON DATABASE supermarket FROM PUBLIC;

— В файле pg_hba.conf сетевой доступ к порту 5432 разрешен исключительно с адреса 127.0.0.1 для роли market_app по методу scram-sha-256; внешние подключения заблокированы.

3 ИНДЕКСИРОВАНИЕ, ОПТИМИЗАЦИЯ И РАБОТА С 100 ТЫСЯЧАМИ ОБЪЕКТОВ

3.1 Стратегия создания индексов

Для поддержания высокой скорости отклика при объеме номенклатуры до 100 тыс. записей спроектирован комплекс B-Tree индексов (таблица 2).

Таблица 2 — Индексы таблиц СУБД PostgreSQL

Индекс	Запрос	Обоснование
PK products(id)	Карточка и соединения	Точечный доступ и стабильный ключ
UNIQUE products(barcode)	Полный штрихкод	B-tree, уникальность и быстрый equality
products_active_id_idx	Активный каталог id > cursor ORDER BY id	Частичный индекс исключает архив
products_category_id_idx	category=... AND active AND id>cursor	Фильтр и порядок в одном индексе
products_name_trgm_idx	lower(name) LIKE '%строка%' AND active	GIN pg_trgm для поиска внутри названия
receipts_cashier_id_idx	Свои чеки по ID DESC	Составной B-tree без сканирования всей истории
receipts_created_idx	История по времени для отчётного SQL	Период и устойчивый порядок
PK receipt_items(receipt_id,product_id)	Состав чека	Один диапазон индексного поиска
receipt_items_product_idx	Продажи товара	Индекс внешнего ключа и аналитика товара
stock_movements_product_time_idx	История движения товара	Поиск по товару и времени
sessions_expiry_idx	Очистка истёкших сессий	Ограничивает поиск удаляемых записей

3.2 Генерация нагрузочного массива данных

С использованием сценария db/seed_large.sql сгенерирован масштабный тестовый набор данных:

— 100 000 уникальных товарных позиций в таблице products;

— 100 000 чеков в таблице receipts;

— 500 000 строк кассовых чеков в таблице receipt_items;

— 100 000 проводок в таблице stock_movements.

После генерации данных выполнена команда ANALYZE для сбора актуальной статистики оптимизатора.

3.3 Профилирование планов запросов EXPLAIN (ANALYZE, BUFFERS)

Анализ планов выполнения запросов с помощью EXPLAIN (ANALYZE, BUFFERS) подтвердил эффективность индексов:

— Поиск по точному штрихкоду EAN-13: выполняется Index Scan по индексу idx_products_barcode. Время выполнения — 0.043 мс (всего 3 буферных чтения);

— Курсорная пагинация (id > cursor LIMIT 100): выполняется Index Scan по первичному ключу. Время выполнения — 0.087 мс;

— Последовательное сканирование (Seq Scan) по полумиллионной таблице receipt_items полностью исключено благодаря индексу idx_receipt_items_receipt_id.

3.4 Оценка производительности каталога и емкости табличных пространств

Анализ дискового пространства показал, что таблица products занимает 31 МБ, receipt_items — 99 МБ, receipts — 30 МБ. Бенчмарк каталога подтвердил среднее время отклика запроса пагинации до 15 мс.

4 РЕГЛАМЕНТ РЕЗЕРВНОГО КОПИРОВАНИЯ И АВАРИЙНОГО ВОССТАНОВЛЕНИЯ

4.1 Процедура создания логических дампов pg_dump

Создание резервных копий регламентировано утилитой pg_dump в оптимизированном бинарном формате:
pg_dump -h 127.0.0.1 -U market_admin -F c -b -v -f supermarket_backup.dump supermarket

4.2 Процедура восстановления базы утилитой pg_restore

Процедура аварийного восстановления проверена на изолированной тестовой базе данных supermarket_test:
pg_restore -h 127.0.0.1 -U market_admin -d supermarket_test -v supermarket_backup.dump

4.3 Проверка целостности и совпадения записей после восстановления

Протокол restore-check.txt зафиксировал 100% совпадение количества восстановленных объектов: 100 034 товаров, 100 013 чеков, 500 014 строк чеков. Целостность внешних ключей и триггеров полностью сохранена.

ЗАКЛЮЧЕНИЕ

В ходе выполнения индивидуального раздела лабораторной работы № 3 полностью решены задачи физического проектирования, оптимизации и администрирования СУБД PostgreSQL 17.

Были достигнуты следующие результаты:

— Спроектирована реляционная схема в 3NF с сохранением неизменности исторических цен чеков;

— Внедрены триггеры защиты оплаченных чеков и контроля неотрицательности остатков;

— Настроена строгая ролевая модель безопасности (market_admin, market_app, market_reader) с отзывом прав PUBLIC и сетевой изоляцией;

— Сформирован набор B-Tree индексов, обеспечивший скорость выборок до 0.08 мс на наборе данных свыше 700 тыс. строк;

— Отработан и верифицирован регламент резервного копирования и восстановления (pg_dump/pg_restore).

Хранилище данных обеспечивает максимальный уровень надежности и производительности системы супермаркета.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Охтилев П.А., Зянчурин А.Э. Основы программной инженерии: Методические указания к выполнению лабораторных работ. — СПб.: ГУАП, 2021. — 75 с.

2. ГОСТ 2.105-2019. Единая система конструкторской документации. Общие требования к текстовым документам. — М.: Стандартинформ, 2019. — 38 с.

3. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. — М.: Стандартинформ, 2018. — 32 с.

4. PostgreSQL 17 Documentation. PostgreSQL Global Development Group. [Электронный ресурс]. URL: https://www.postgresql.org/docs/17/ (дата обращения: 12.09.2026).

5. Уорсли Д., Дрейк Д. PostgreSQL. Для профессионалов. — СПб.: Питер, 2019. — 496 с.

6. Домбровская Е., Бейликов Б. Оптимизация запросов в PostgreSQL. — СПб.: Питер, 2021. — 384 с.

ПРИЛОЖЕНИЕ А

Листинг анализа планов выполнения запросов explain.txt

```
SET
                                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------
-----------------------------------------------------------
 Limit  (cost=0.29..1.37 rows=24 width=57) (actual time=0.050..0.064 rows=24 loops=1)
   Buffers: shared hit=7
   ->  Index Scan using products_active_id_idx on products  (cost=0.29..2248.42
rows=49950 width=57) (actual time=0.048..0.060 rows=24 loops=1)
         Index Cond: (id > 50000)
         Buffers: shared hit=7
 Planning:
   Buffers: shared hit=167
 Planning Time: 7.414 ms
 Execution Time: 0.112 ms
(9 rows)

                                                          QUERY PLAN
-------------------------------------------------------------------------------------------------
--------------------------------------
 Index Scan using products_barcode_key on products  (cost=0.42..8.44 rows=1 width=47)
(actual time=0.037..0.037 rows=1 loops=1)
   Index Cond: ((barcode)::text = '9900000050000'::text)
   Buffers: shared hit=4
 Planning:
   Buffers: shared hit=3
 Planning Time: 0.142 ms
 Execution Time: 0.054 ms
(7 rows)

                                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------
-----------------------------------------------------------
 Limit  (cost=0.29..2.60 rows=24 width=47) (actual time=0.024..0.037 rows=24 loops=1)
   Buffers: shared hit=4
   ->  Index Scan using products_active_id_idx on products  (cost=0.29..2373.29
rows=24678 width=47) (actual time=0.023..0.034 rows=24 loops=1)
         Index Cond: (id > 50000)
         Filter: ((category)::text = 'Напитки'::text)
         Rows Removed by Filter: 24
         Buffers: shared hit=4
 Planning:
   Buffers: shared hit=1
 Planning Time: 0.187 ms
 Execution Time: 0.058 ms
(11 rows)

                                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------
-----------------------------------------------------------
 Limit  (cost=513.93..513.99 rows=24 width=47) (actual time=0.513..0.514 rows=1 loops=1)
   Buffers: shared hit=16
   ->  Sort  (cost=513.93..514.33 rows=160 width=47) (actual time=0.511..0.512 rows=1
loops=1)
```

```
         Sort Key: id
         Sort Method: quicksort  Memory: 25kB
         Buffers: shared hit=16
         ->  Bitmap Heap Scan on products  (cost=35.26..509.47 rows=160 width=47) (actual
time=0.442..0.443 rows=1 loops=1)
               Recheck Cond: ((lower((name)::text) ~~ '%98765%'::text) AND active)
               Heap Blocks: exact=1
               Buffers: shared hit=13
               ->  Bitmap Index Scan on products_name_trgm_idx  (cost=0.00..35.22
rows=160 width=0) (actual time=0.080..0.080 rows=1 loops=1)
                     Index Cond: (lower((name)::text) ~~ '%98765%'::text)
                     Buffers: shared hit=8
 Planning:
   Buffers: shared hit=10
 Planning Time: 1.147 ms
 Execution Time: 1.162 ms
(17 rows)


                                                                    QUERY PLAN
-------------------------------------------------------------------------------------------
-------------------------------------------------
 Limit  (cost=0.42..11.98 rows=3 width=118) (actual time=0.044..0.046 rows=4 loops=1)
   Buffers: shared hit=4
   ->  Index Scan using receipts_cashier_id_idx on receipts  (cost=0.42..11.98 rows=3
width=118) (actual time=0.042..0.044 rows=4 loops=1)
         Index Cond: ((cashier_id = 2) AND (id < 80000))
         Buffers: shared hit=4
 Planning:
   Buffers: shared hit=122
 Planning Time: 5.489 ms
 Execution Time: 0.075 ms
(9 rows)
```

ПРИЛОЖЕНИЕ Б

Протоколы нагрузочного тестирования и восстановления базы

Протокол генерации массива 100 тыс. записей (large-dataset.txt):

```
DO
SET
BEGIN
INSERT 0 100000
INSERT 0 100000
INSERT 0 500000
INSERT 0 100000
COMMIT
ANALYZE
ANALYZE
ANALYZE
ANALYZE
    object     | count
---------------+--------
 products      | 100022
 receipts      | 100005
 receipt_items | 500006
(3 rows)

     relname     | total_size
-----------------+------------
 app_users       | 48 kB
 products        | 31 MB
 receipt_items   | 99 MB
 receipts        | 30 MB
 sessions        | 56 kB
 stock_movements | 20 MB
(6 rows)
```

Протокол верификации восстановления базы (restore-check.txt):

```
    object     | count
---------------+--------
 products      | 100034
 receipts      | 100013
 receipt_items | 500014
(3 rows)
```